Desenvolvimento de Apps para Mobile

Fabrício Tonetto Londero

Aula 01



Quem sou?

- **Fabrício Tonetto Londero**
- **E-mail**: fabricio.londero@facens.br
- **GitHub**: https://github.com/Ernakh/
- **LinkedIn**: https://www.linkedin.com/in/fabriciolondero
- **Website**: https://linktr.ee/fabriciolondero

FABRÍCIO LONDERO





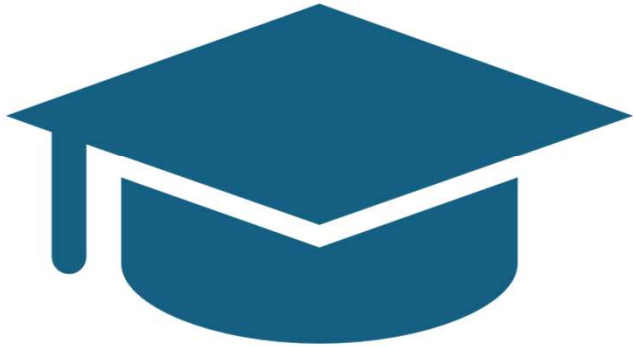






Formação acadêmica

- Doutorado em Educação – UFSM – concluído em 2024
- Mestrado em Ciência da Computação – UFSM – concluído em 2016
- Especialização em Gestão de Banco de Dados – ULBRA – concluído em 2015
- Especialização em Aplicações Web – FURG – concluído em 2015
- Programa Especial de Graduação de Formação de Professores para a Educação Profissional (PEG) – UFSM – 2016
- Graduação em Sistemas de Informação – UNIFRA – concluído em 2012
- Técnico em Informática – FASCLA – concluído em 2010

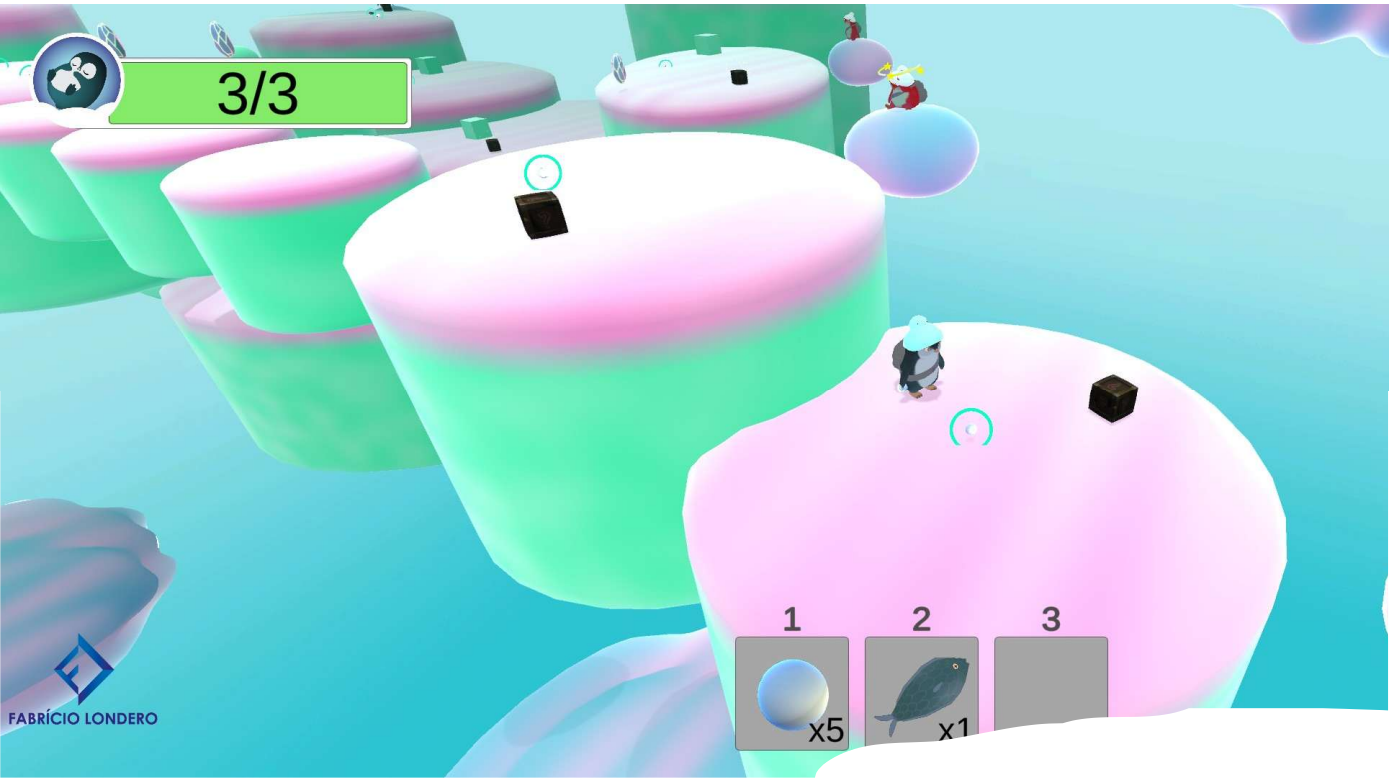


Experiência profissional

- 2009-2010: Abille Soluções em TI – Analista de Suporte de Sistemas
- 2011-2015: Analista de Sistemas – UNIFRA
- 2014-2018: Professor – FISMA
- 2021-2021: Professor Universitário – Unochapecó
- 2017 – 2024: Professor Universitário - UFN
- 2021 – 2024: Professor – Parceira entre Atos e UFN – 13 academias (Java e .Net) ~400 alunos
- 2025 - atualmente: Professor Universitário - Facens

Experiência profissional

- 08/2017 até 12/2024: UFN – Universidade Franciscana
  - Professor dos cursos de Sistemas de Informação, Jogos Digitais e Ciência da Computação
  - Coordenador de Tecnologias Educacionais no setor UFN Digital
  - Coordenador do curso de Sistemas de Informação desde julho de 2022
  - Membro dos NDE dos cursos de Sistemas de Informação, Jogos Digitais e Ciência da Computação
  - Ministrei disciplinas em cursos de Especialização



QUIZ GLACIAL

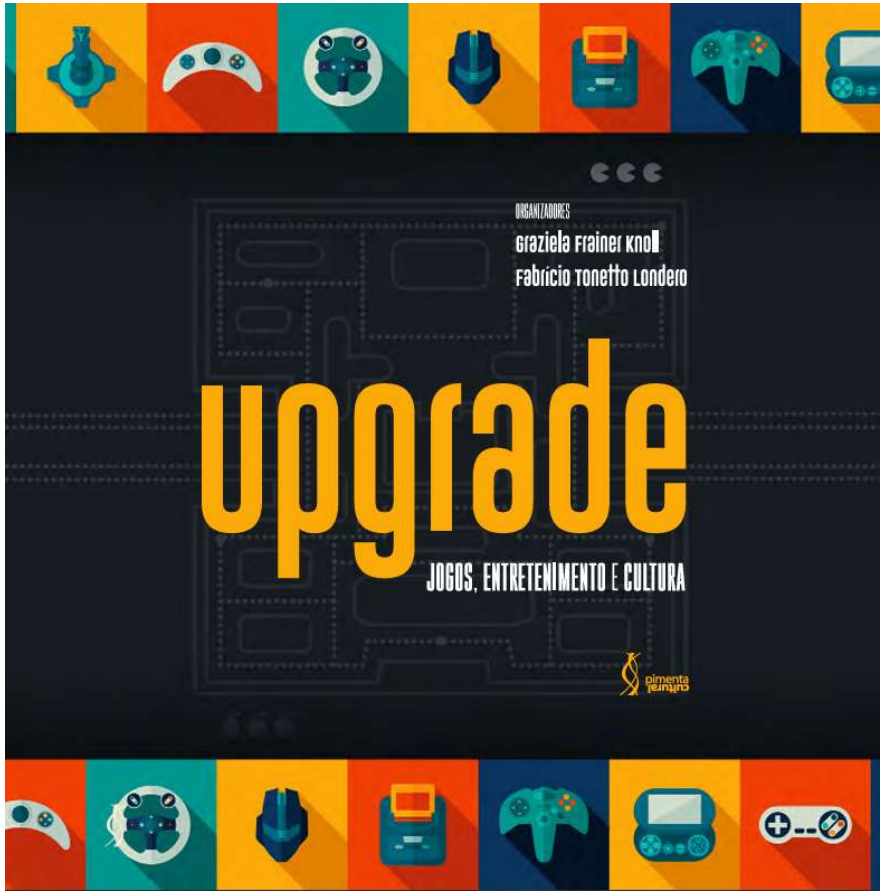
Projetos Relevantes

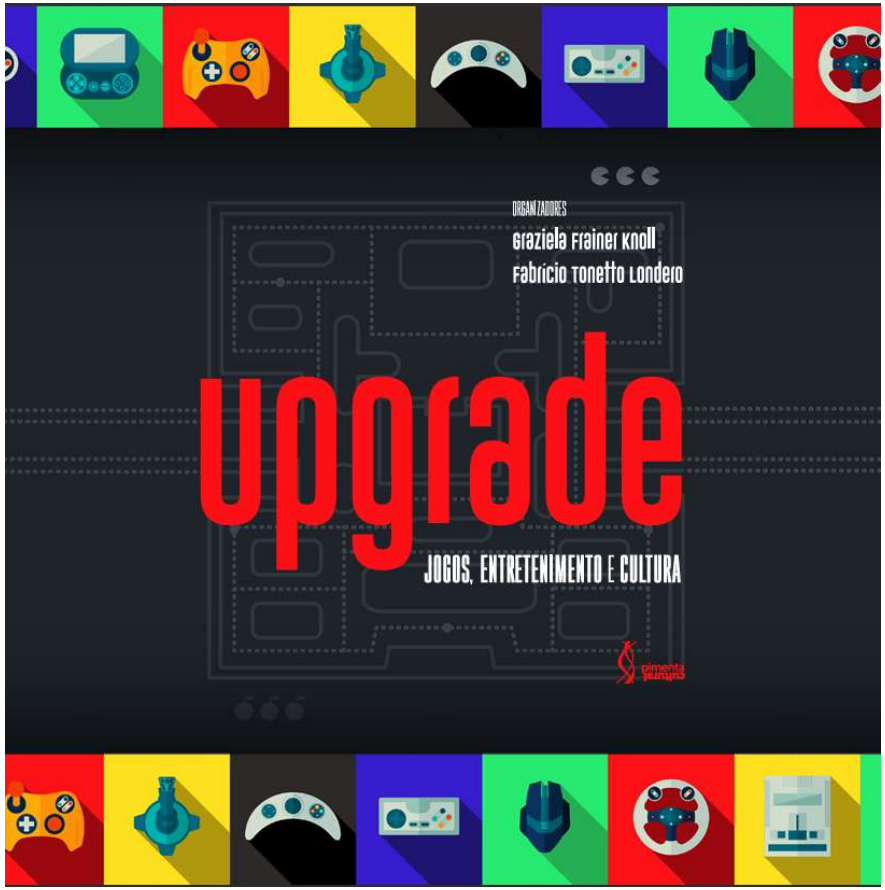



Os pinguins brincalhões me acorrentaram aqui. Por favor, vença-os e me liberte das correntes.


QUIZ GLACIAL
Jogar
Sou Professor
Instruções
Créditos
Sair
FABRÍCIO LONDERO

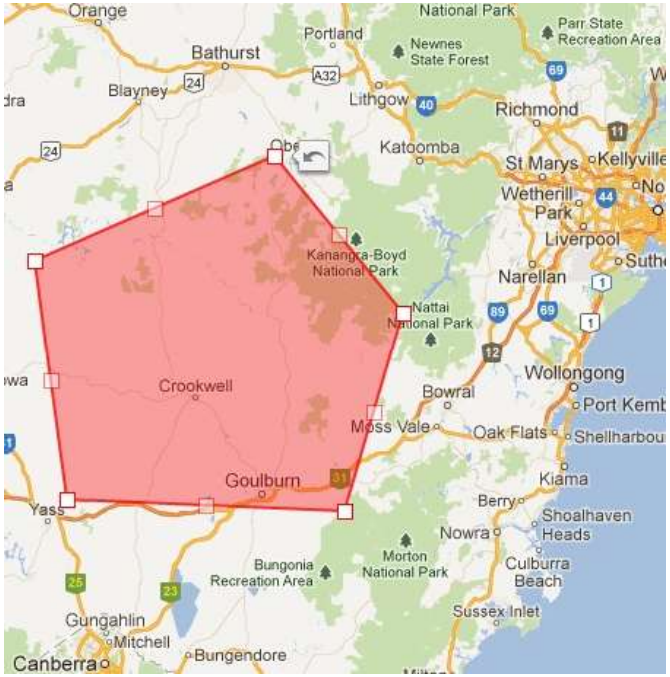

QUIZ GLACIAL
FIM DE JOGO!
Acertos: 7
Erros: 3
Tempo: 261,91 segundos
Sair
FABRÍCIO LONDERO


ORGANIZADORES
Graziela Frainer Knoll
Fabrício Tonetto Londero
upgrade
JOGOS, ENTRETENIMENTO E CULTURA
pimenta cultural


ORGANIZADORES
Graziela Frainer Knoll
Fabrício Tonetto Londero
upgrade
JOGOS, ENTRETENIMENTO E CULTURA
pimenta cultural

TCC 2012

- OPÇÃO DE MARKETING PARA SISTEMAS ANDROID POR MEIO DE GEOLOCALIZAÇÃO
- https://periodicos.ufn.edu.br/index.php/disciplinarumNT/article/view/1365



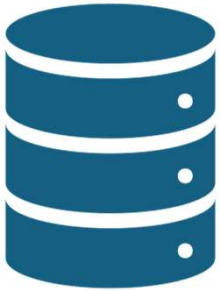




MAQUINA VIRTUAL DALVIK



Recebe os arquivos .dex dos compiladores



Compacta o .dex e outros recursos em um arquivo .apk

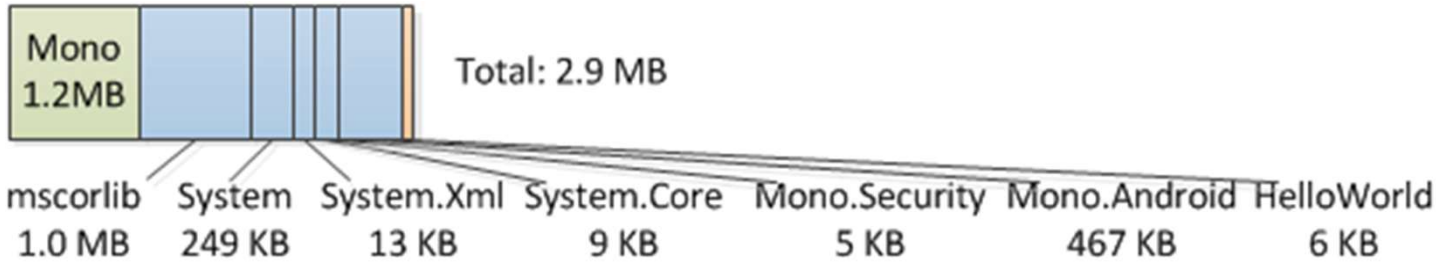
TCC 2012

- OPÇÃO DE MARKETING PARA SISTEMAS ANDROID POR MEIO DE GEOLOCALIZAÇÃO
  - MonoDroid

Tamanho para aplicação com bibliotecas completas:



Aplicação após coleta de lixo (funcionalidades não utilizadas das bibliotecas):



Expectativas dos alunos

- Mapa da turma
  - Cidades
  - Times
- Faixa etária
- Quem trabalha na área
  - E diretamente com mobile
- Conhecimento prévio
- Expectativas para a disciplina

Apresentação da disciplina

- DESENVOLVIMENTO DE APPS PARA MOBILE
  - Plano de Aula

Objetivo

- Transmitir os princípios básicos e boas práticas de desenvolvimento de softwares para dispositivos móveis, familiarizar o aluno com o sistema operacional e framework Android e adquirir experiência prática com a programação para dispositivos móveis.

Matriz

- Relaciona-se diretamente com outras disciplinas do curso
  - CONSTRUÇÃO DE ALGORITMOS
  - ENGENHARIA DE SOFTWARE
  - PROGRAMAÇÃO ORIENTADA À OBJETOS
  - BANCO DE DADOS
  - ANÁLISE DE SISTEMAS
  - ...

Mercado

- Fornece a base necessária para compreensão e aplicação de conceitos avançados no desenvolvimento de aplicativos móveis.
- Possui forte conexão com as demandas do mercado de trabalho, preparando os alunos para desafios práticos na carreira.
- Possibilidade de aplicação em diferentes contextos profissionais, como desenvolvimento de aplicativos para empresas, startups e projetos autônomos.
- Formação de uma base sólida para especialização futura e crescimento na área.

Mercado



Desenvolvedor Mobile - Salário Brasil

R$ 5.636,62/mês

Atualizado em 05 de janeiro de 2025

Hoje um **Desenvolvedor Mobile** ganha em média R$ 5.636,62 para uma jornada de trabalho de 42 horas semanais de acordo com pesquisa do **Portal Salario** junto a dados de 53.674 profissionais admitidos e desligados em regime CLT nos últimos 12 meses divulgados pelo **Novo CAGED**.

Em 2025 a remuneração para Desenvolvedor Mobile pode variar entre o piso salarial mínimo de R$ 5.482,68 e o teto salarial de R$ 12.569,81, dependendo do segmento da empresa, localidade, formação, experiência na função e política de cargos e salários da empresa.

Salário por estado

AM BA CE DF ES GO MG
MS MT PB PE PR RJ RN
RS SC SE SP

Salário por cidade

São Paulo Porto Alegre Barueri
Belo Horizonte Rio de Janeiro Curitiba
Brasília Florianópolis Passo Fundo
Blumenau

Tópicos dessa pesquisa salarial:

Descrição Histórico salarial Salário por região
O que faz Tabela salarial Demanda



Desenvolvedor Mobile - Estado de São Paulo

R$ 6.772,53/mês

Atualizado em 05 de janeiro de 2025

Um **Desenvolvedor Mobile** trabalhando no estado de São Paulo ganha em média **R$ 6.772,53** para uma jornada de trabalho de 41 horas semanais.

A **faixa salarial** do Desenvolvedor Mobile fica entre **R$ 5.476,00** salário mediana da pesquisa e o teto salarial de **R$ 14.903,90**, sendo que **R$ 6.587,56** é a média do piso salarial 2025 de acordos coletivos levando em conta profissionais em regime CLT de todo o estado de SP.

Tópicos dessa pesquisa salarial:

Histórico salarial O que faz Tabela salarial
Demanda Empresas que + contratam
Carga horária Salários por nível e porte da empresa
Piso salarial Salário PCDs Dúvidas trabalhistas
Gráficos salariais Folha de pagamento
CBO: 3171-10

Salário por cidade

São Paulo Barueri Campinas
São José dos Campos São Bernardo do Campo
Ribeirão Preto Americana Hortolândia

- https://www.salario.com.br/profissao/desenvolvedor-mobile-cbo-317110/

Dilemas

- Criar sites mobiles ou app?
- Utilizar aplicações nativas ou híbridas?
- Android ou iOS?

O que é computação móvel?



O que é computação móvel?





O que é computação móvel?







Conceito

“Acesso a informação em qualquer lugar e a qualquer momento”

Conceito



“Levar um computador e todos os arquivos e softwares necessários para onde precisar”



“É a capacidade de se utilizar um dispositivo computacional mesmo estando em movimento”



“É uma variedade de dispositivos sem fio que tem a mobilidade de permitir as pessoas a se conectarem a internet, proporcionando transmissão sem fio e acesso a informação de onde quer que ela esteja”

Principais características



Dispositivos **pequenos** e **portáteis;**



Execução de aplicações locais ou distribuídas;



Usuários em movimento.

Evolução Histórica



Informação



Poder:

a informação é poder! (*há séculos*)



Mais poder ainda - a informação pode ser acessada:

Por qualquer pessoa

Em qualquer lugar

A qualquer momento



A mobilidade está em:

Dispositivos

Redes de comunicação

Evolução Histórica

- 1876: Alexander Graham Bell (1847–1922) inventa o telefone
- 1ª chamada telefônica: *"Mr. Watson, come here, I want to see you"*

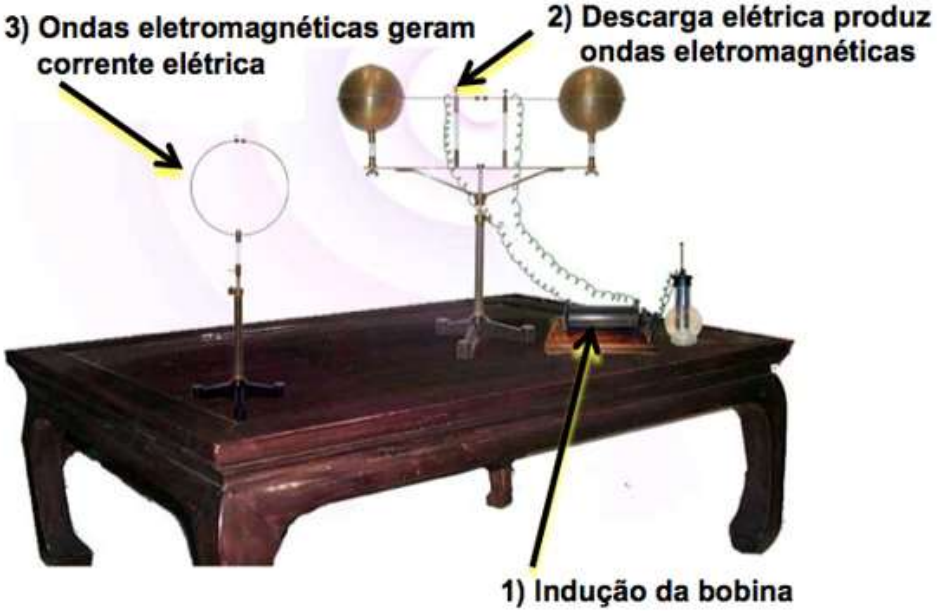
- Entretanto, o Congresso dos Estados Unidos reconheceu, em 2002, que o aparelho foi inventado pelo italiano Antonio Meucci, por volta de 1860, que o chamou de “telégrafo falante”



Evolução Histórica

1887: Henrich Rudolph Hertz (1857–1894) detecta as ondas eletromagnéticas (Previstas pelas equações de Maxwell).

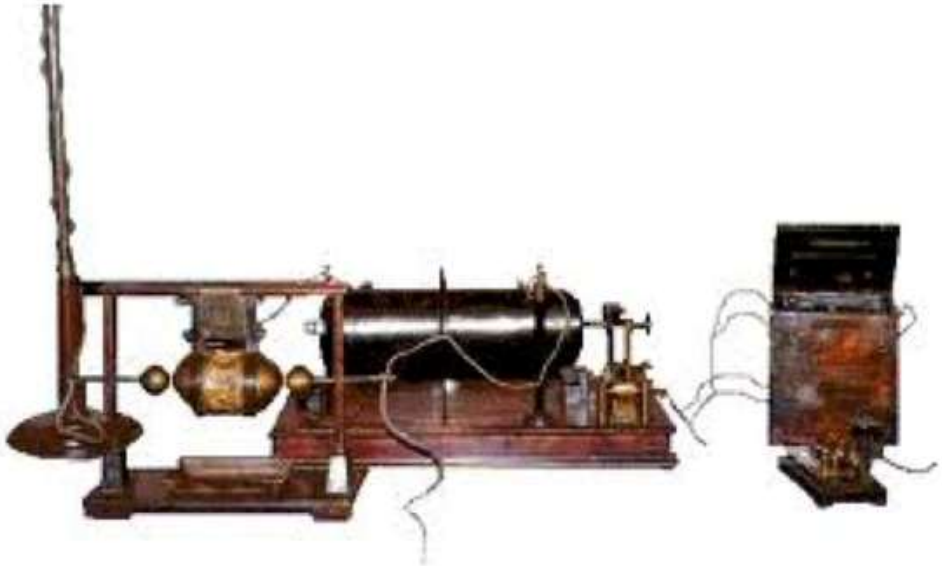




Evolução Histórica

1896: Guglielmo Marconi (1874–1937) inventa o primeiro receptor sem fio prático: o telégrafo sem fio.

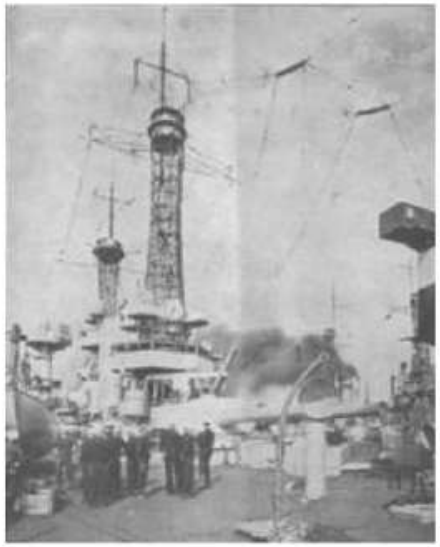




Evolução Histórica

1907: Início do serviço de radiodifusão comercial transatlântico.

- Estações terrestres imensas
- Antenas de 30 × 100 m



Evolução Histórica

1914: Início da Primeira Guerra Mundial

- Rápido desenvolvimento de comunicações
- Interceptações (no ar e sem codificação é para todos)





Evolução Histórica

1921: Radiodifusão comercial entra em operação nos Estados Unidos.





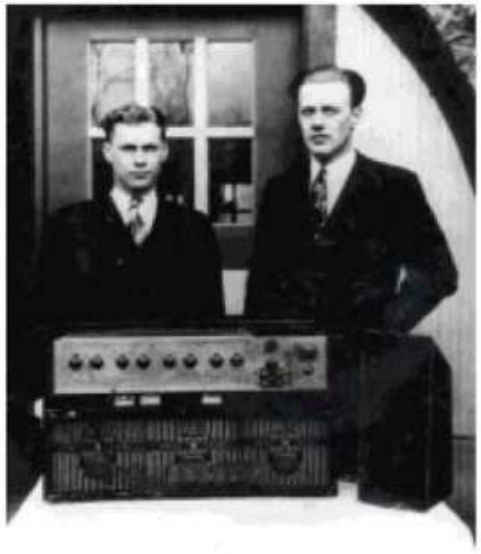
Evolução Histórica

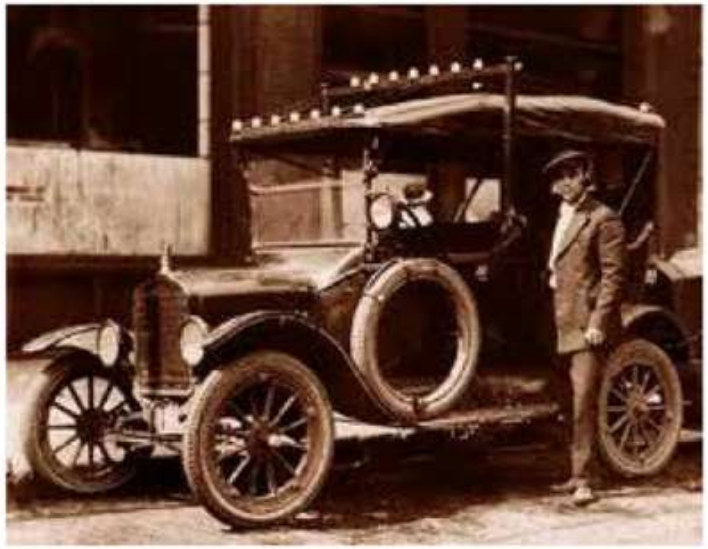
- Sendo que a primeira emissora comercial dos Estados Unidos foi a WEAF, 1921.
- No Brasil, a primeira emissora de rádio foi a Rádio Sociedade do RJ, criada em 20 de abril de 1923



Evolução Histórica

1928: A Polícia de Detroit introduz um sistema de acionamento de carros baseado em radiodifusão (unidirecional) na faixa de 2 MHz.

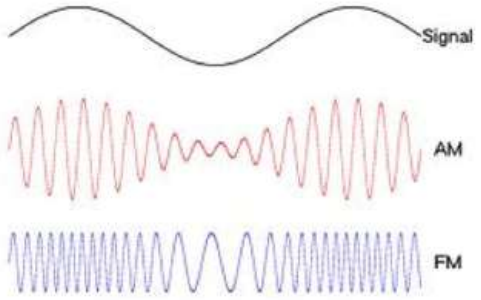




Evolução Histórica

1935: Modulação em Frequência - FM (*Frequency Modulation*) surge como alternativa para a Modulação em Amplitude - AM (*Amplitude Modulation*)

- Reduz os problemas de ruídos na transmissão
- Melhor desempenho com relação a perda de sinal



- A modulação em frequência (FM) transmite informações através de uma portadora que varia sua frequência instantânea, ao contrário da modulação em amplitude (AM), em que a amplitude da portadora varia enquanto a frequência permanece constante.

- Em FM, o comprimento de onda é de alguns centímetros, logo, objetos desse tamanho são capazes de refletir as ondas e estão menos sujeitas a interferências (ruídos);
  - Apesar disso, ela é uma onda muito mais penetrante e não se reflete bem nas camadas baixas da atmosfera.

- Em AM, o comprimento das ondas são na ordem de metros, conseguindo usar muito bem as camadas da atmosfera para se refletir, alcançando distâncias mais longas;

- Ondas FM necessitam de repetiras, caso contrário só alcançam até a linha do horizonte, enquanto de AM não necessitam de repetidoras pois se propagam muito bem na atmosfera e alcançam longas distâncias.

Evolução Histórica

1939: Evolução de pesquisas e uso da comunicação via rádio durante a Segunda Guerra Mundial.





*Mulheres trabalhando na recepção de mensagens.

Evolução Histórica

1945: AT&T Bell Labs inicia experimentos no uso de frequências mais altas.

- Objetivo era melhorar os serviços móveis.

Existem diferentes faixas de frequências que são utilizadas para os mais diversos fins:

- **GSM** (800/1900 MHz)
- **Wi-Fi** (2,4 GHz)
- **Bluetooth** (2,4 GHz)
- **ZigBee** (2,4 GHz)

Evolução Histórica

1947: AT&T lança o IMTS (*Improved Mobile Telephone Service*)

- Um sistema de transmissão em que apenas uma torre de alta potência atendia uma grande área ou cidade
- Em seguida, AT&T Bell Labs propõe o conceito celular



Evolução Histórica

Anos 50: com o uso de transistores, os equipamentos reduzem de tamanho e podem ser transportáveis.

- Surgem os primeiros sistemas de *paging* (*pager*)



Evolução Histórica

Anos 60: Bell Labs testa as técnicas de comunicação celular e surgem os primeiros aparelhos portáteis.





Alguns pesavam até 5 Kg.

Evolução Histórico

Anos 70: AT&T lança o sistema celular conhecido por AMPS (*Advanced Mobile Phone System*).

- Era um serviço de luxo
- Destinado para uso em automóveis
- Utilização limitada (baixa autonomia da bateria)
- 1ª rede celular no mundo foi lançada no Japão (1979)





Evolução Histórico

1983: o DynaTAC, da Motorola, recebe aprovação da FCC (*Federal Communications Commission*) para ser vendido no mercado.





Evolução Histórica

- 1983: Começam a surgir outros sistemas ao redor do mundo, mas o AMPS em uso nos EUA, Brasil e grande parte do mundo ainda era considerado um sistema de primeira geração
- 1991: Validação inicial dos padrões nos EUA
  - CDMA (Code Division Multiple Access)
    - Acesso Múltiplo por Divisão de Código
  - TDMA (Time Division Multiple Access)
    - Acesso Múltiplo por Divisão de Tempo



Evolução Histórica

1992: Introdução do sistema celular Pan-Europeu GSM
(*Groupe Spéciale Mobile*)

Atualmente *Global System for Mobile Communications*

1992: Nokia lança seu primeiro modelo de celular GSM
- Modelo 1011



Evolução Histórica

Foi o primeiro dispositivo com o toque padrão *Nokia Tune*

1994: Nokia vende 20 milhões de unidades do modelo 2110



Evolução Histórica

1996: Motorola lança o StarTAC, um celular com apenas 88 gramas.



Evolução Histórica

- 1998: ativação dos primeiros celulares digitais em São Paulo. Surge a ANATEL — órgão regulador das telecomunicações no País.
- 2000: torna-se disponível no Brasil acesso a Internet via celular — WAP (Wireless Application Protocol).
- 2002: Nokia lança o primeiro celular com câmera fotográfica, o 7650.
- 2006: o número de usuários chega a 2,6 bilhões em todo o Mundo. 100% das pessoas, em países como Finlândia e Itália, possuem celular.

Evolução Histórica

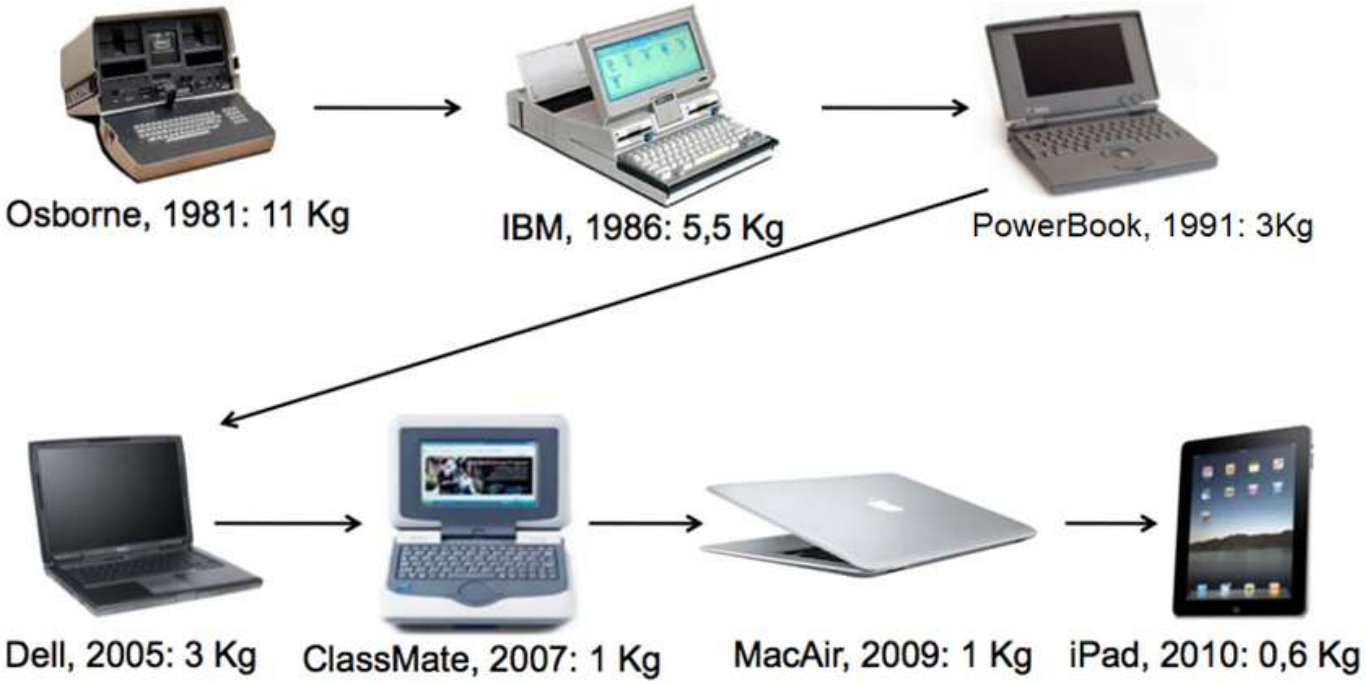
2007: sistema GSM domina o mercado mundial com uma fatia de 80%

De 2007 até agora, a evolução foi extremamente significativa
Principalmente em dispositivos e serviços:





Evolução Histórica



Evolução Histórica



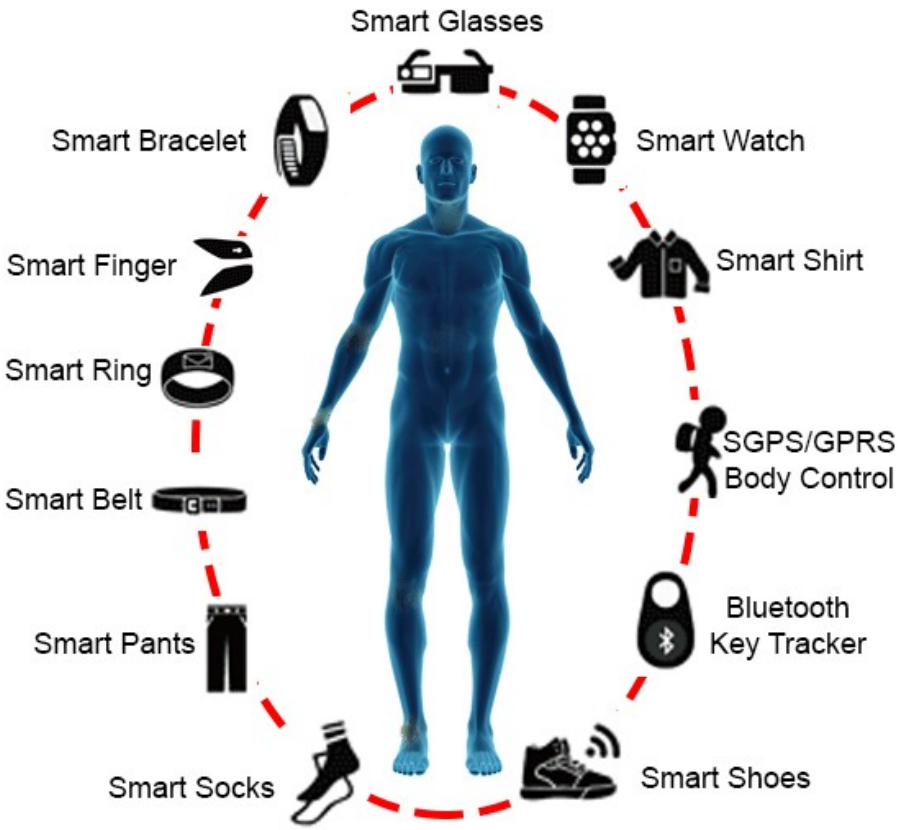
Evolução Histórica



Evolução Histórica



Evolução Histórica







Evolução Histórica

Lei de Cooper:

"O número de transmissões pelo ar dobra a cada 30 meses..."

- Capacidade atual de transmissão = 1 trilhão de vez a mais que a inicial.

- Crescimento igual nos próximos 60 anos
  - Com a tecnologia já conhecida

Martin Cooper, *inventor do telefone celular*



Mobilidade

Evolução Natural

Lucro de desenvolvedores para mobile

Antes dos Smartphones





Smartphones

- 30% para a loja (Google Play, Apple Store)
- 70% para os desenvolvedores



Apple

- 99 dólares anuais
- Custo elevado para começar a desenvolver (MacBook + iPhone + Tablet)



Google Play

- Mais barato que Apple para começar
  - PC
  - Smartphone
  - Tablet

- 25 dólares para poder publicar (1x somente)

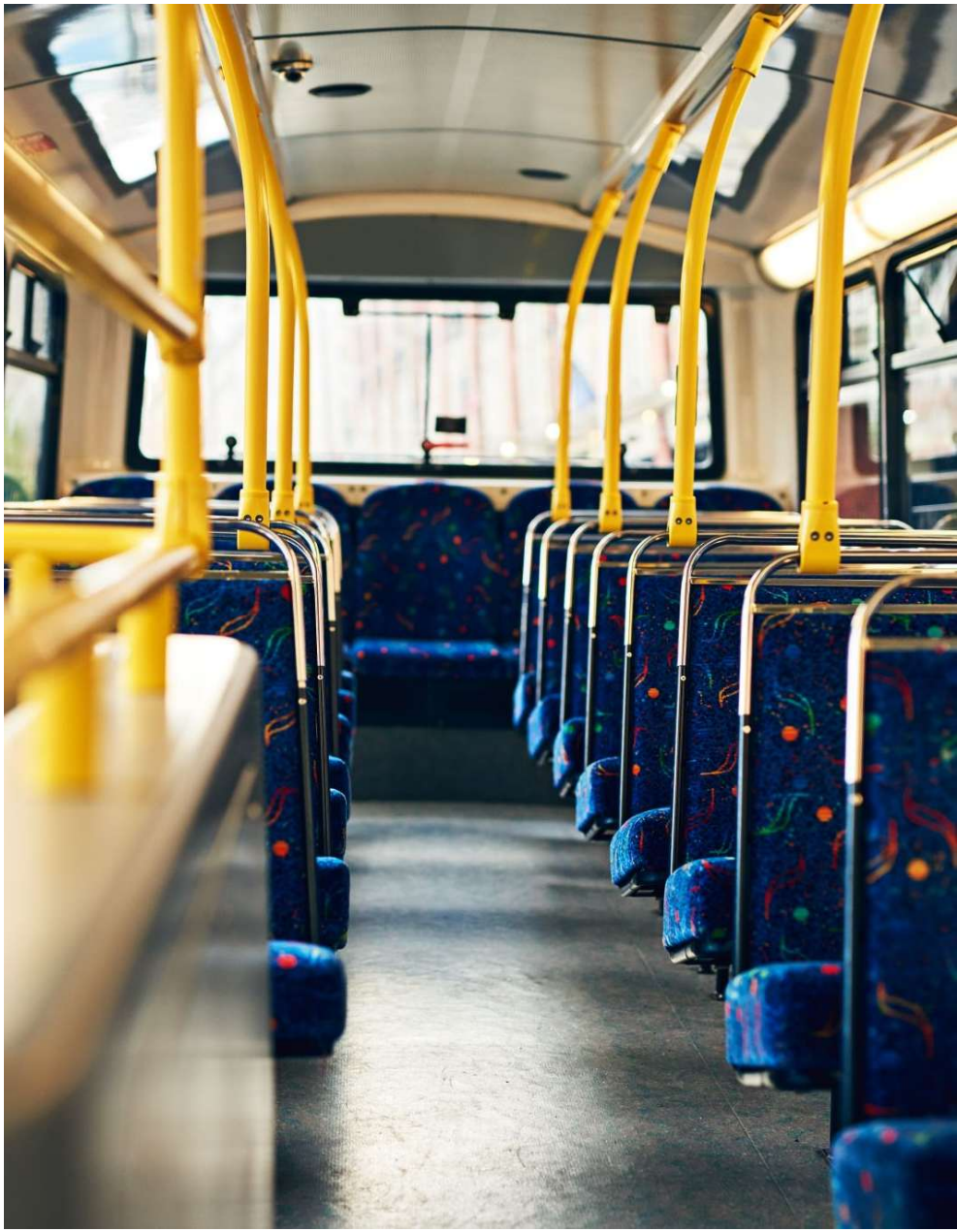


Curiosidade

- Maioria dos smartphones pelo mundo são Android
- Culturalmente, os maiores lucros para desenvolvedores são da Apple
- Segundo a Escola Brasileira de Games:
  - Em um mesmo aplicativo para as duas plataformas, 89% do lucro é proveniente da plataforma Apple
  - Diferente se for propagandas embutidas em aplicativos

Google Play

- Muito fácil
- Qualquer pessoa pode se cadastrar como desenvolvedor
- Bom para iniciantes
- Devido a facilidade tem muitos produtos, e fica difícil de você ser encontrado
- Mecanismo de busca recebe muitas reclamações



Google Play

A plataforma não entrega o teu produto para todos os usuários, eles terão que procurar pelo teu jogo ou um marketing externo precisa ser aplicado.

- Download de forma orgânica dificilmente será suficiente

A plataforma vai entregar aos usuários os aplicativos/jogos que estão em alta

- Performando bem
- Promovidos

Ideal para aprender o processo como um todo da publicação de um app, assim como à fazer o próprio marketing.

Google Play

Qual modelo de negócio funciona melhor?

Freemium | Free with ADS

Culturalmente falando, o usuário da plataforma tem o hábito de baixar as coisas de graça – Hábito de consumo

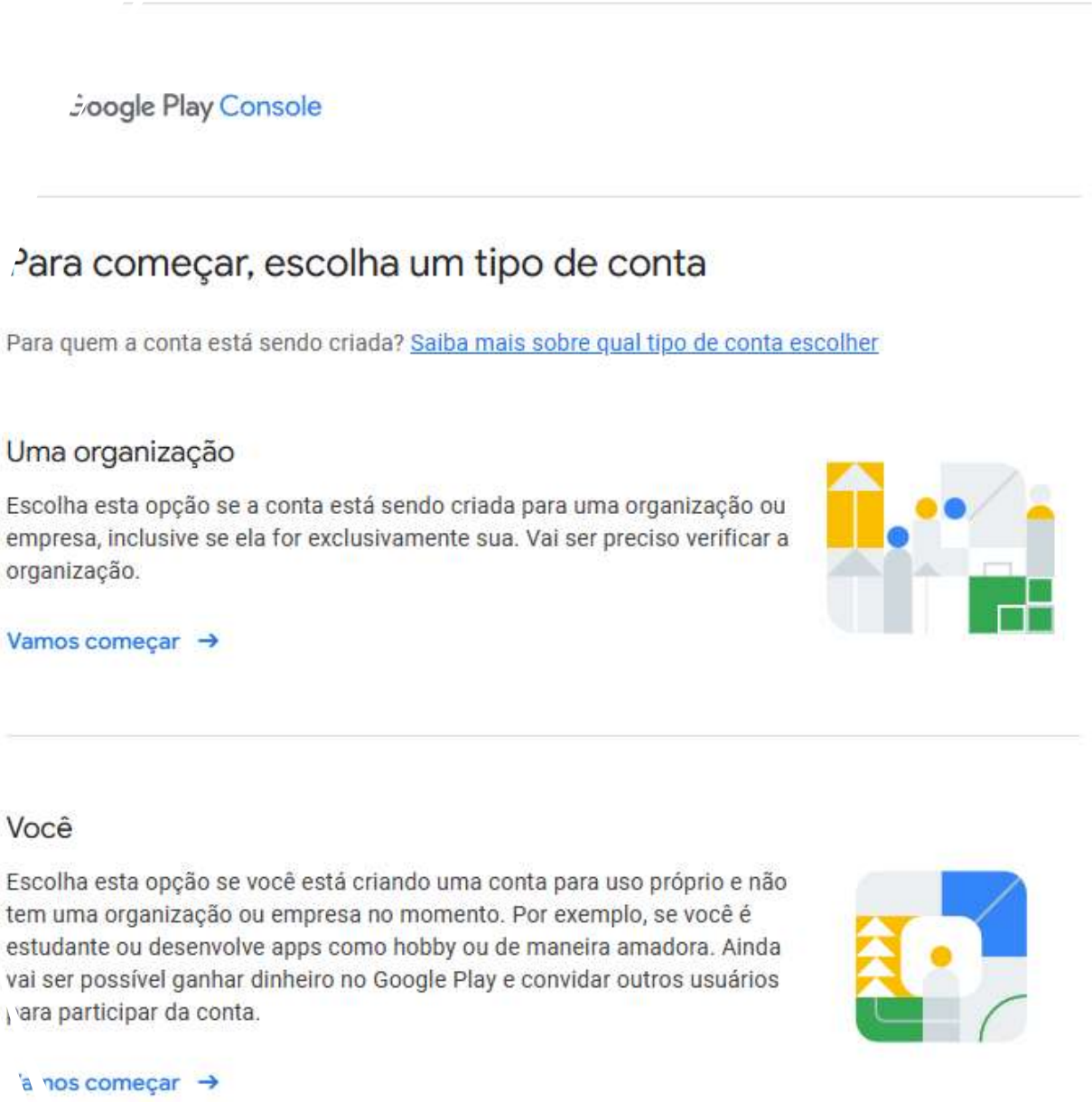
Um app pago pode não performar muito bem

Existem exceções



Google Play

- Como publicar?
  - [Apps Android no Google Play](https://play.google.com/console) (https://play.google.com/console)



Google Play Console



Do que você precisa ao criar sua própria conta de desenvolvedor

Dados obrigatórios para criar a conta:

Um endereço de e-mail para usuários do Google Play
Ele vai aparecer no Google Play para que os usuários possam entrar em contato com você. Você vai precisar receber um código para verificação. Saiba mais sobre as verificações para desenvolvedores

Um número de telefone e um endereço de e-mail de contato com o Google
Eles só serão usados pelo Google para entrar em contato com você se necessário, e não vão ser exibidos no Google Play. Você vai precisar receber um código para verificação.

Uma forma de pagamento
Pagar a taxa de registro única de USD 25

É preciso confirmar sua identidade para publicar apps

Voltar Continuar

Google Play Console



Primeiro, informe o nome do desenvolvedor

É o que os usuários vão ver no Google Play para representar seu perfil de desenvolvedor e seus apps. O nome não precisa ser igual ao seu nome e pode ser alterado posteriormente.

Insira o nome de uma conta de desenvolvedor 0 / 50

Voltar Próximo

Google Play Console



Vincule um perfil para pagamentos para confirmar sua identidade

Para criar uma conta de desenvolvedor, é necessário vincular um perfil para pagamentos do Google. Se você não tiver, crie um.

O que é um perfil para pagamentos?
O perfil para pagamentos é usado em todos os produtos do Google. Você gerencia suas informações em um só lugar e poupa tempo. Saiba mais

Como o Google Play usa seu perfil para pagamentos
Um perfil para pagamentos é fundamental para a criação e a verificação de uma conta de desenvolvedor do Play Console. O Google Play usa seu perfil para pagamentos para coletar e verificar seu nome comercial e endereço. Saiba mais

Criar ou selecionar perfil para pagamentos

Voltar Próximo

Google Play

Após o pagamento, o painel de publicação é liberado

Jogos e aplicativos ilimitados

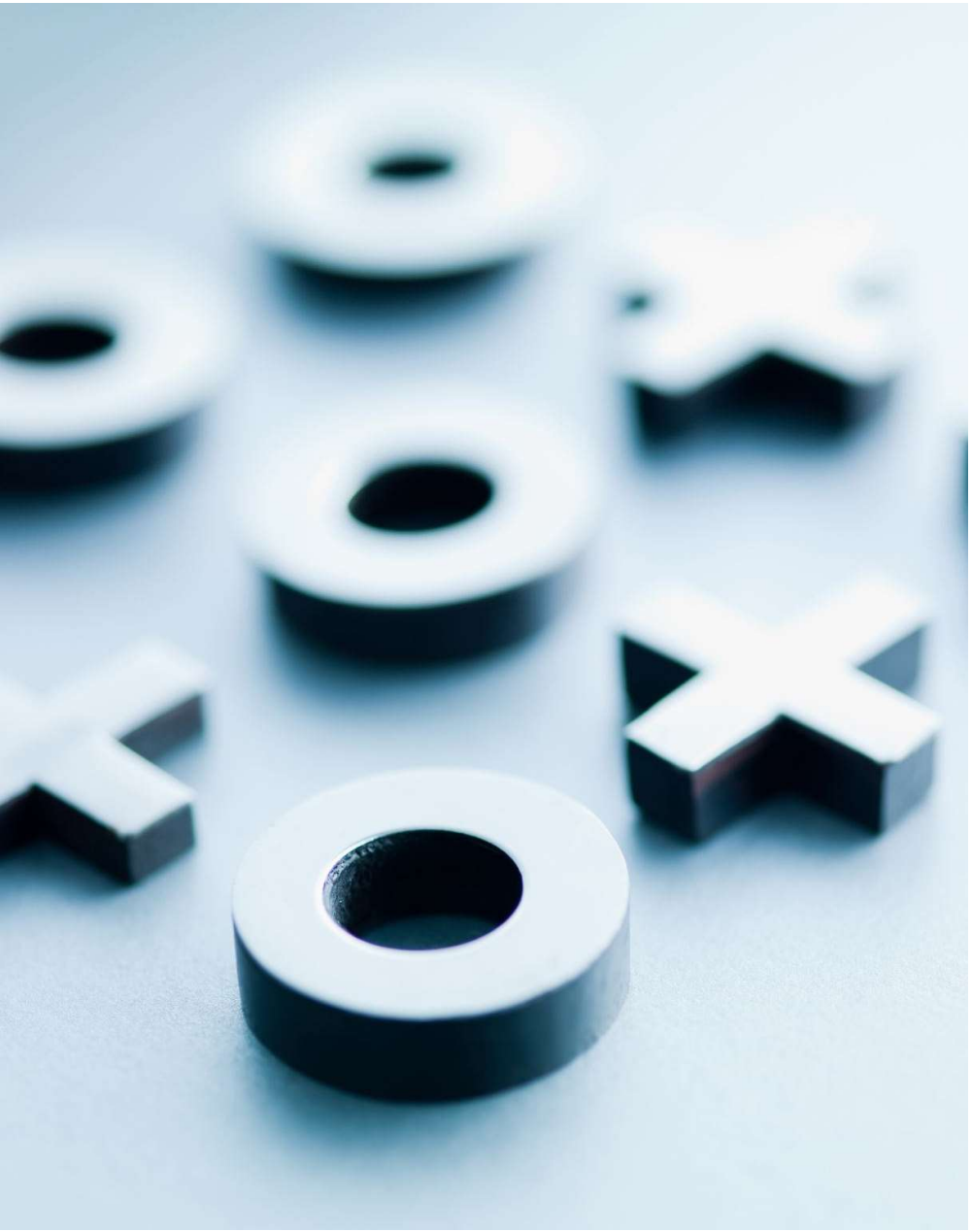
Por tempo indeterminado

Taxa única para frear um pouco aplicativos faltos e maliciosos



Apple Store

- Curadoria muito mais rígida que a Google Play
- Prezem por aplicativos com mais qualidade
- Revisão sistemática do produto – seguem diretrizes
- Baixa tolerância com conteúdo que eles julgam inapropriado ou ofensivo
- Logo, é mais difícil ter um app lançado
  - As vezes requer adaptar o conteúdo do app/jogo
  - Sim, imagens, textos...
  - Mudar ou conversar com o suporte



Apple Store

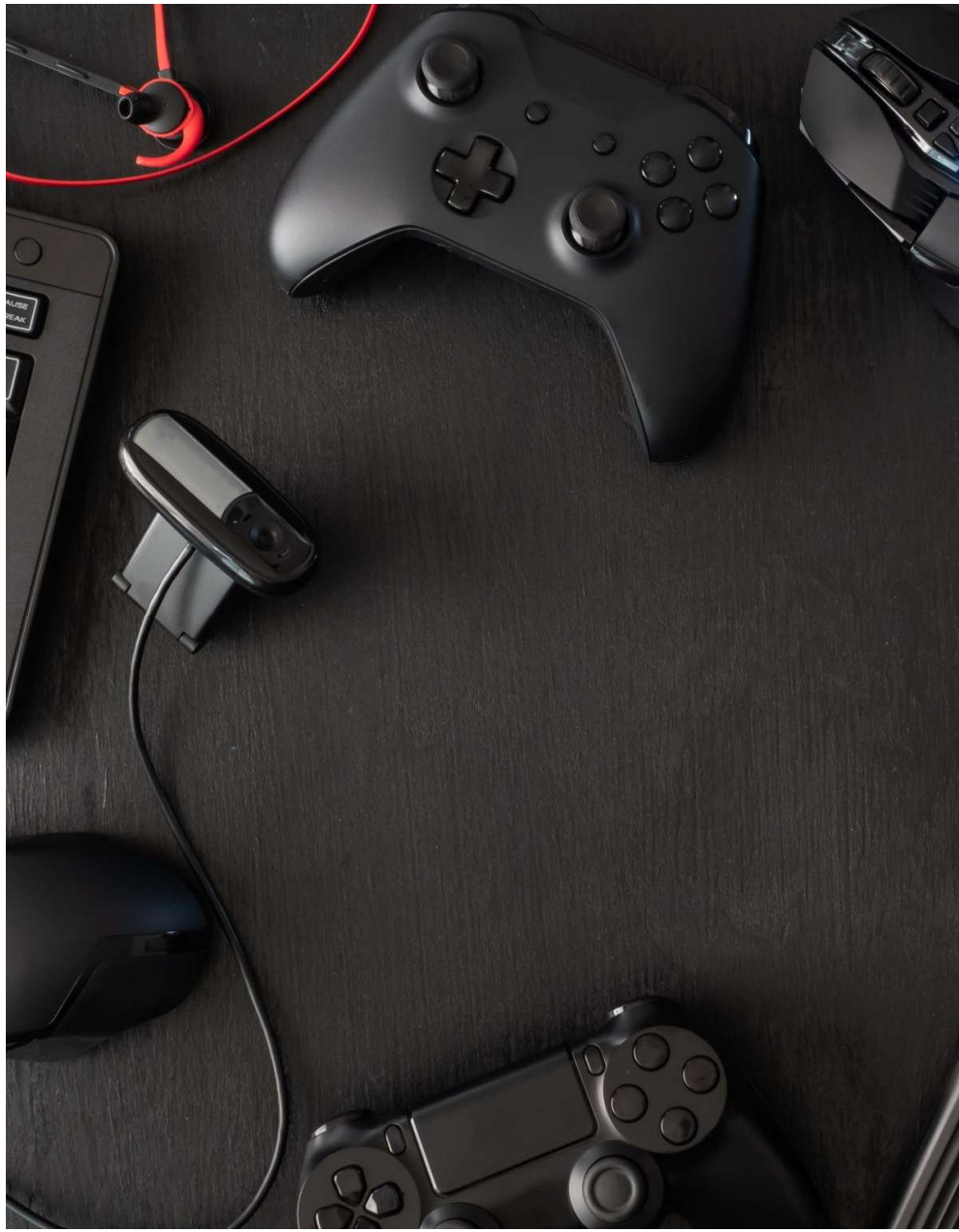
- Reconhecido por ter produtos de mais qualidade
- Hábitos de consumo diferente da Google Play
  - Existe uma cultura diferente, de pagar mais por aquilo que julga-se ser bom
  - Logo, apps premium são mais bem aceitos
- A descoberta de aplicativos é melhor
  - Mesmo assim, ainda existem algoritmos que listam os apps em alta, os melhores avaliados e os promovidos.

Apple Store

- Modelo de negócios
  - Premium
  - Freemium
  - Free with ADS

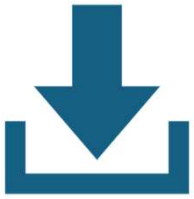
Apple Store

- Desenvolver é caro
  - É necessário equipamentos da Apple para desenvolver e testar os jogos/aplicativos
  - Taxa anual de 99 dólares
- Registrar-se em developer.apple.com
- Suporte decente!



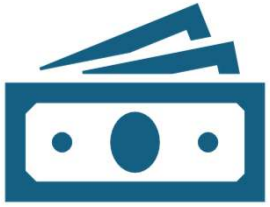
Google Play Store		App Store
US$ 25 - Taxa Única	Taxa Desenvolvedor	US$ 99 - Taxa Anual
15%	Taxa de Serviço nas transações	15%
Freemium / Free with ADS	Modelos de Negócio	Premium / Free to Play
	Processo de publicação	
play.google.com/console	Endereço para inscrição	developer.apple.com/programs/enroll

Reportagens



Google Play tem mais downloads que App Store:

https://canaltech.com.br/apps/volume-de-downloads-do-google-play-e-tres-vezes-maior-que-o-da-app-store-173056/



Devs ganham mais dinheiro na App Store:

https://canaltech.com.br/apps/google-play-vs-app-store-devs-ganham-muito-mais-dinheiro-na-loja-da-maca-142189/

Java

- Java é uma linguagem de programação de alto nível e uma plataforma de computação desenvolvida pela Sun Microsystems (agora propriedade da Oracle).
- Foi criada por James Gosling e sua equipe em meados da década de 1990. Uma das principais características do Java é sua portabilidade, o que significa que os programas escritos em Java podem ser executados em qualquer dispositivo que tenha uma máquina virtual Java (JVM) instalada, independentemente do sistema operacional subjacente.

Java



Orientação a objetos: Java é uma linguagem de programação orientada a objetos, o que significa que todos os elementos do programa são objetos ou classes. Isso permite uma organização mais modular e reutilizável do código.



Portabilidade: Como mencionado anteriormente, os programas Java são altamente portáteis devido à JVM. Isso significa que um programa Java compilado pode ser executado em qualquer dispositivo que tenha uma JVM compatível instalada, desde computadores de mesa até dispositivos móveis e até mesmo embarcados.



Segurança: Java foi projetado com uma série de recursos de segurança, incluindo sandboxing (caixa de areia) para executar código não confiável de forma segura, verificação de bytecode para detectar e prevenir vulnerabilidades de segurança e gerenciamento de memória automatizado para evitar corrupção de memória e bugs relacionados.



Multi-threading: Java oferece suporte nativo à programação multi-threaded, permitindo que os desenvolvedores criem aplicativos que executem várias tarefas simultaneamente. Isso é útil para criar aplicativos que respondem rapidamente a eventos do usuário e fazem uso eficiente dos recursos do sistema.



Java Development Kit (JDK)

- É um conjunto de ferramentas que os desenvolvedores usam para desenvolver aplicativos Java.
- Ele inclui o compilador Java, que traduz o código-fonte Java em bytecode executável, e a JVM, que executa o bytecode em tempo de execução.
- Além disso, o JDK contém uma variedade de ferramentas de desenvolvimento, como depuradores, geradores de documentação e bibliotecas padrão para facilitar o desenvolvimento de aplicativos Java.
- Em resumo, o JDK é essencial para escrever, compilar e executar programas Java.

Maven

- Maven é uma ferramenta de automação de construção de projetos amplamente utilizada na comunidade de desenvolvimento de software Java.
- Ele fornece uma maneira eficiente de gerenciar a construção, relatórios e documentação de projetos, além de facilitar a gestão de dependências.
- Aqui estão os principais componentes e conceitos do Maven:
  - **POM (Project Object Model):** No coração do Maven está o arquivo POM, que é um arquivo XML que descreve o projeto e suas configurações. O POM contém informações como versões de dependências, plugins a serem usados, configurações de compilação, testes e empacotamento do projeto.
  - **Convenção sobre Configuração:** Maven segue o princípio de "convenção sobre configuração", o que significa que muitas configurações padrão são assumidas com base em convenções estabelecidas. Isso reduz a necessidade de configuração manual e facilita o uso do Maven em projetos comuns.
  - **Gestão de Dependências:** Uma das características mais poderosas do Maven é sua capacidade de gerenciar as dependências do projeto de forma automática. Você pode especificar as dependências do seu projeto no arquivo POM e o Maven cuidará de baixar as bibliotecas necessárias e suas transições de forma transparente.

Maven

- Aqui estão os principais componentes e conceitos do Maven: (continuação)
  - **Ciclo de Vida de Construção:** Maven define um ciclo de vida de construção padrão, que consiste em fases como compilação, teste, empacotamento e implantação. Cada fase do ciclo de vida é associada a um conjunto de objetivos predefinidos, que são executados em uma determinada ordem.
  - **Plugins:** Maven é altamente extensível através de plugins. Existem plugins para realizar uma variedade de tarefas, como compilar código, executar testes, gerar relatórios, empacotar artefatos, publicar em repositórios e muito mais. Os plugins podem ser configurados no arquivo POM e ativados conforme necessário.
  - **Repositórios Maven:** Maven faz uso de repositórios remotos e locais para armazenar e recuperar dependências e artefatos. Existem repositórios públicos amplamente utilizados, como o Maven Central, que contém milhares de bibliotecas Java prontas para uso.
- Em resumo, o Maven simplifica o processo de construção e gerenciamento de projetos Java, fornecendo uma estrutura consistente e automatizada para lidar com dependências, compilação, teste e distribuição de software. Ele promove boas práticas de desenvolvimento, como a reutilização de componentes e a padronização da estrutura do projeto

Gradle

- O Gradle é uma ferramenta de automação de construção de projetos que oferece uma abordagem flexível e poderosa para o gerenciamento de projetos e construção de software.
- Ele foi desenvolvido para simplificar o processo de construção, permitindo que os desenvolvedores descrevam a estrutura do projeto e as dependências de forma declarativa, usando uma DSL (Domain-Specific Language) baseada em Groovy ou Kotlin.

Maven x Gradle

- A escolha entre Maven e Gradle depende de várias considerações, incluindo as necessidades específicas do projeto, preferências da equipe de desenvolvimento e familiaridade com as ferramentas.
  - Se o projeto é grande e complexo, com requisitos personalizados de construção, o Gradle pode ser mais adequado devido à sua flexibilidade e capacidade de lidar com cenários complexos de construção. Por outro lado, se o projeto é mais simples e segue as convenções do Maven, o Maven pode ser uma escolha mais direta.
  - Se você prefere uma abordagem mais "opinionada" e baseada em convenções, o Maven pode ser a escolha certa, pois possui uma estrutura mais rígida e orientada por padrões. Por outro lado, se você prefere uma abordagem mais flexível e personalizável, o Gradle oferece mais liberdade para configurar o projeto de acordo com suas necessidades específicas.

Maven x Gradle

- A escolha entre Maven e Gradle depende de várias considerações, incluindo as necessidades específicas do projeto, preferências da equipe de desenvolvimento e familiaridade com as ferramentas. (Continuação)
  - O Gradle é geralmente mais rápido do que o Maven, especialmente em projetos grandes, devido à sua execução incremental e capacidade de paralelização. Se o desempenho é uma consideração importante para o seu projeto, o Gradle pode ser uma escolha preferencial.
  - Maven tem uma vasta base de usuários e uma grande quantidade de plugins disponíveis, tornando mais fácil encontrar soluções para problemas comuns e integrar-se a outras ferramentas. Se o suporte da comunidade e a disponibilidade de plugins são importantes para você, o Maven pode ser mais atraente nesse aspecto.

Programação Orientada a Objetos

Cenário da POO:

- Redução do tempo de desenvolvimento de SW;
- Redução dos custos do projeto;
- Atendimento da demanda:
  - Gerada pela evolução tecnológica
- Reutilização de código;
- Facilidade de manutenção.

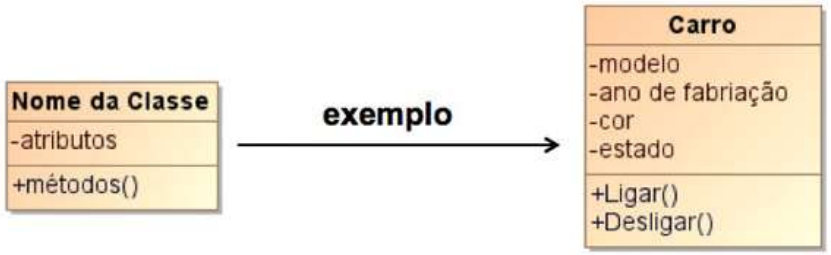
Programação Orientada a Objetos

Em orientação a objetos, uma classe é uma estrutura que abstrai um conjunto de objetos com características similares.

A classe define o comportamento dos objetos através de métodos e atributos

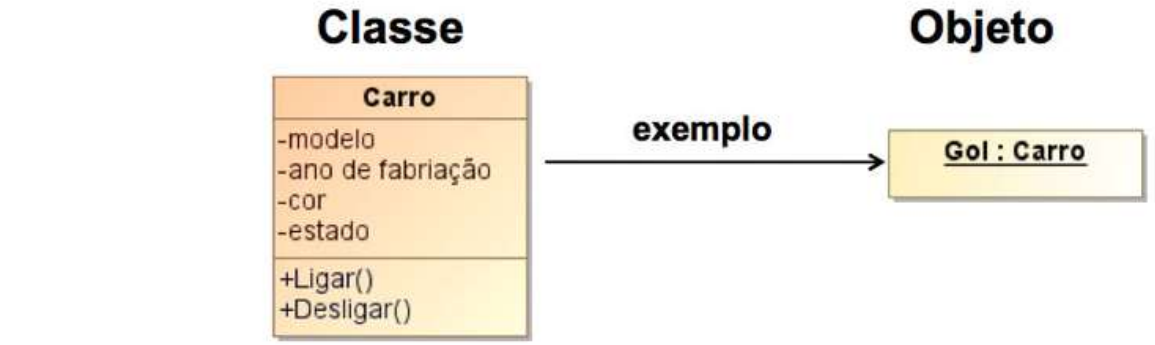
- **Métodos** = funções na linguagem estruturada
- **Atributos** = variáveis na linguagem estruturada

A classe descreve as características e funcionalidades dos objetos

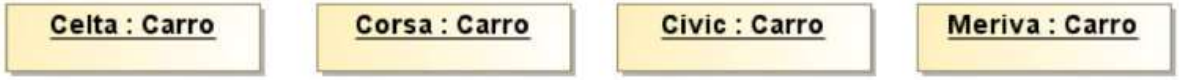


Programação Orientada a Objetos

O **objeto** é a **instância** de uma **classe**
Ex: Classe Carro e Objeto Gol



É possível ter vários objetos a partir de uma classe



Encapsulamento

É o empacotamento dos atributos e métodos numa classe

- **Proteção** dos dados:
  - Público (+)
  - Privado (-)
  - Protegido (#)



RESUMO

Classe: representa um conjunto de objetos
Subclasse: classe filha que herda os atributos e os métodos da classe mãe (Herança)
Objeto: instância de uma classe
Atributo: características do objeto
Método: funcionalidades do objeto
Mensagem: troca de informação entre os objetos
Associação: utilização de recursos entre objetos
Abstração: classe não instanciável
Polimorfismo: métodos com muitas formas: sobrecarga, sobrescrita

Classes em JAVA

- Uma classe que represente uma **pessoa** em Java poderia ser especificada da seguinte forma:

```
public class Pessoa {


}
```

- Perguntas:
  1. Qual o nome da classe?
  2. O que significa o “public” ?
  3. Ela possui atributos e métodos?
  4. Como deve ser salvo o arquivo referente a esta classe?

Atributos de uma classe

- Os atributos de uma classe são adicionados dentro da própria classe;
- Os atributos irão representar as características do objeto;
- Podem receber valores iniciais ou não;

```
public class Pessoa {

    String nome;
    int idade;
    double salario = 0;
}
```

- Perguntas:
  1. Quais atributos a classe Pessoa possui?
  2. Quais os valores iniciais de cada atributo?
  3. Estes atributos estão encapsulados?

Encapsulamento de atributos

- O encapsulamento dos atributos de uma classe servem para proteger certas informações da mesma;
- O encapsulamento de um atributo pode ser: **`private`**, **`public`** e **`protected`**;

```
public class Pessoa {

    private String nome;
    private int idade;
    private double salario = 0;
}
```

- Perguntas:
  1. Qual a diferença entre **`private`**, **`public`** e **`protected`**?
  2. Os atributos da classe Pessoa são acessíveis de outras classes?
  3. Como podemos acessar um atributo privado?

Métodos de uma classe

- Um método de uma classe especifica uma funcionalidade da mesma;
- Os métodos também podem ser encapsulados;

```
public class Pessoa {

    private String nome;
    private int idade;
    private double salario = 0;

    public void andar(){
    }

    public void sentar(){
    }
}
```

Perguntas:

1. Os métodos `andar` e `sentar` podem ser acessados por outras classes?
2. Os métodos `andar` e `sentar` recebem parâmetros?
3. Os métodos `andar` e `sentar` retornam informações?

Parâmetros de um método

- Utilizamos parâmetros quando desejamos passar uma informação para um objeto de uma classe através de um método;

```
public class Pessoa {

    private String nome;
    private int idade;
    private double salario = 0;

    public void setNome(String nome){
        this.nome = nome;
    }
}
```

Perguntas:

1. O que o método **setNome** faz?
2. Que parâmetros ele recebe?
3. O que significa a palavra "**this**" ?

Retorno de um método

- Quando um método precisa retornar uma informação para o objeto que chamou o mesmo, precisamos definir qual o tipo de retorno desta informação

```
public class Pessoa {

    private String nome;
    private int idade;
    private double salario = 0;

    public String getNome(){
        return this.nome;
    }

}
```

Perguntas:

1. O que o método **getNome** faz?
2. Que parâmetros ele recebe?
3. Que tipo de dado ele retorna?

Atributos/Variáveis de um método

- Um método também poderá possuir atributos locais que serão utilizados pelo mesmo para alguma finalidade;

```
public class Pessoa {

    private String nome;
    private int idade;
    private double salario = 0;

    public void calculaIdade(int anoNascimento){
        int anoAtual = 2014;
        this.idade = anoAtual - anoNascimento;
    }
}
```

Setters e *Getters*

- São os métodos seletores e modificadores dos atributos de uma classe;

- Através deles aplica-se a regra do encapsulamento, protegendo as informações internas da classe;

- Métodos “get”: modificadores
- Métodos “set”: seletores

Setters e *Getters* de Pessoa

```
public class Pessoa {

    private String nome;
    private int idade;
    private double salario = 0;

    public String getNome() {
        return nome;
    }
    public void setNome(String nome) {
        this.nome = nome;
    }
    public int getIdade() {
        return idade;
    }
```

```
public void setIdade(int idade) {
        this.idade = idade;
}
public double getSalario() {
        return salario;
}
public void setSalario(double salario)
{
        this.salario = salario;
}

}
```

Classes Pessoa e Carro

- Uma classe pode conter atributos que são objetos de outras classes

```
public class Pessoa {

    private String nome;
    private Carro c;

}
```



```
public class Carro{

    private String placa;
    private int ano;
    private String modelo;
    private String marca;
}
```

Perguntas:

1. Quantos carros cada pessoa possui?

Métodos construtores

- Um método construtor possui as instruções que serão executadas sempre que instanciarmos um objeto de uma classe;

- Um método construtor possui o **mesmo nome da classe** e não possui **nenhum retorno**:
  - Note que **nenhum retorno** é diferente de **void!**

- Podemos ter mais de um método construtor na mesma classe, diferenciando-os nos parâmetros que os mesmos recebem;

Método construtor

```
public class Pessoa {
    private String nome;
    private Carro c;

    public Pessoa(){
    }
    public Pessoa(String nome){
        this.nome = nome;
        c = null;
    }
    public Pessoa(String nome, Carro c){
        this.nome = nome;
        this.c = c;
    }
}
```

Perguntas:

1. Quantos construtores a classe Pessoa possui?
2. Qual a diferença entre eles?
3. Porquê os construtores são públicos?

Método construtor

```
public class Pessoa {

    private String nome;
    private Carro c;

}
```

Perguntas:

1. Quantos construtores a classe Pessoa possui?

Herança

- Muitas vezes diferentes classes possuem características em comum;
- Para evitar a criação de uma nova classe com todas as características repetidas, podemos fazer uso das características de uma classe já existente através de **herança**;
- Através da herança, uma classe irá herdar todas as características de uma outra classe (métodos, atributos, etc.);

Herança

```
class Eletrodomestico {
    private boolean ligado;
    private int voltagem;
    private int consumo;

    public Eletrodomestico(boolean ligado, int voltagem, int consumo) {
        this.ligado = ligado;
        this.voltagem = voltagem;
        this.consumo = consumo;
    }
}
```

Herança

```
class TV extends Eletrodomestico {
    private int canal;
    private int volume;
    private int tamanho;

    public TV(int voltagem, int consumo, int canal, int volume, int
tamanho) {
       this.canal = canal;
      this.volume = volume;
      this.tamanho = tamanho;
    }
}
```

Instanciando objetos

- Um objeto é uma instância de uma classe;
- Para instanciar um objeto de uma classe, utilizamos a palavra **new**;
- Quando instanciamos um objeto, o construtor do mesmo é executado;

Instanciando objetos

```
public class Pessoa {
    private String nome;
    private Carro c;

    public Pessoa(){
    }
    public Pessoa(String nome){
        this.nome = nome;
        c = null;
    }
    public Pessoa(String nome, Carro c){
        this.nome = nome;
        this.c = c;
    }
}
```

```
public class Carro
{
        private String placa;
        private int ano;
        private String modelo;
        private String marca;
}
```

Instanciando objetos das classes Pessoa e Carro:

```
Carro c = new Carro();

Pessoa p1 = new Pessoa();
Pessoa p2 = new Pessoa("Fabrício");
Pessoa p3 = new Pessoa("Felipe",c);
```